

- This chapter considers how to compute gradients efficiently
- Also how to initialize parameters.
 - ↳ If gradients are too large/small it implodes the training process.

7.1: Problem Definitions:

- Consider the network:

$$\begin{aligned}
 h_1 &= a[B_0 + \Omega_0 x] && \# 3 \text{ hidden layers} \\
 h_2 &= a[B_1 + \Omega_1 h_1] && \# \text{Multivariate input } x \\
 h_3 &= a[B_2 + \Omega_2 h_2] && \# B_n \text{ are bias vectors} \\
 &&& \# \Omega_n \text{ are weight matrices} \\
 f(x, \theta) &= B_3 + \Omega_3 h_3
 \end{aligned}$$

- Also have individual loss terms L_i , which is negative log-likelihood of the ground truth label y_i given model prediction $f(x_i, \theta)$.

↳ The total loss is the sum of these terms over the training data:

$$L[\theta] = \sum_{i=1}^I L_i$$

- The most commonly used training algorithm is SGD, which updates parameters as:

$$\theta_{t+1} \leftarrow \theta_t - \alpha \sum_{i \in B_t} \frac{\partial L_i[\theta_t]}{\partial \theta}$$

- To compute the update rule, we need the derivatives:

$$\frac{\partial L_i}{\partial B_k} \quad \text{and} \quad \frac{\partial L_i}{\partial \Omega_k}$$

for parameters $\theta = \{B_k, \Omega_k\}$ at every layer $k \in \{0, 1, \dots, K\}$ AND for each index i in the batch.

↳ Done with the backpropagation algorithm!

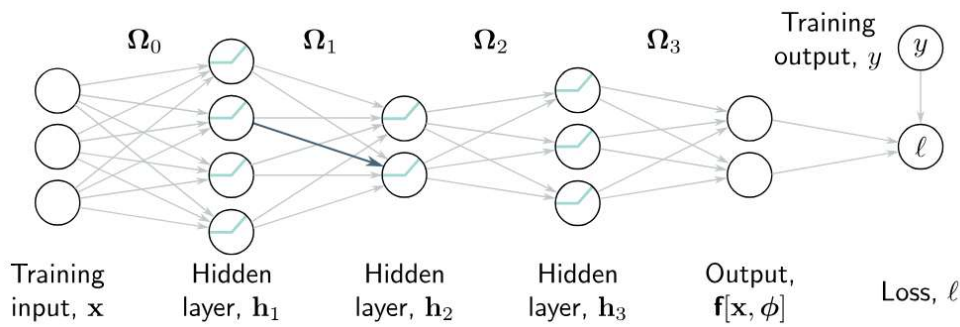


Figure 7.1 Backpropagation forward pass. The goal is to compute the derivatives of the loss ℓ with respect to each of the weights (arrows) and biases (not shown). In other words, we want to know how a small change to each parameter will affect the loss. Each weight multiplies the hidden unit at its source and contributes the result to the hidden unit at its destination. Consequently, the effects of any small change to the weight will be scaled by the activation of the source hidden unit. For example, the blue weight is applied to the second hidden unit at layer 1; if the activation of this unit doubles, then the effect of a small change to the blue weight will double too. Hence, to compute the derivatives of the weights, we need to calculate and store the activations at the hidden layers. This is known as the *forward pass* since it involves running the network equations sequentially.

7.2: Computing Derivatives:

- The derivative of the loss tells us how the loss changes when we make a small change to the parameters.
- Optimization algorithms exploit this to minimize loss.
- Intuition behind backpropagation:

#1:

Each weight (element of Ω_k) multiplies the activation at a source hidden unit; adds the result to a destination hidden unit in the next layer.

So a small change in the weight is amplified/attenuated by the activation at the source hidden unit.

Thus we run the network for each data example in the batch; store the activations of all the hidden units.

This is called a forward pass! The stored activations will be used to compute the gradients.

#2:

A small change in a weight/bias causes a ripple effect of changes through the subsequent network. The change is made which changes the value of the destination hidden unit, which changes the values of the next layer of hidden units, which eventually changes the model output, thus the loss.

So to know how changing a parameter modifies the loss, we need to know how changes to every subsequent hidden layer will modify their successor.

These quantities are needed when considering other parameters in the same/earlier layers.

So we can calculate them once, then reuse them.

Example:

• To calculate how a change in weight/bias feeding into h_i modifies the loss, we need to know:

Example:

- To calculate how a change in weight/bias feeding into h_3 modifies the loss, we need to know:
 - 1) how a change in layer h_3 changes model output f
 - 2) how a change in model output changes loss $\mathcal{L}$
- To calculate how a change in weight/bias feeding into h_2 modifies the loss, we need to know:
 - 1) how a change in h_2 affects h_3
 - 2) how h_3 changes model output
 - 3) how output changes loss
- To calculate how a change in weight/bias feeding into h_1 modifies the loss, we need to know:
 - 1) how a change in h_1 affects h_2
 - 2) how a change in h_2 affects h_3
 - 3) how h_3 changes model output
 - 4) how output changes loss

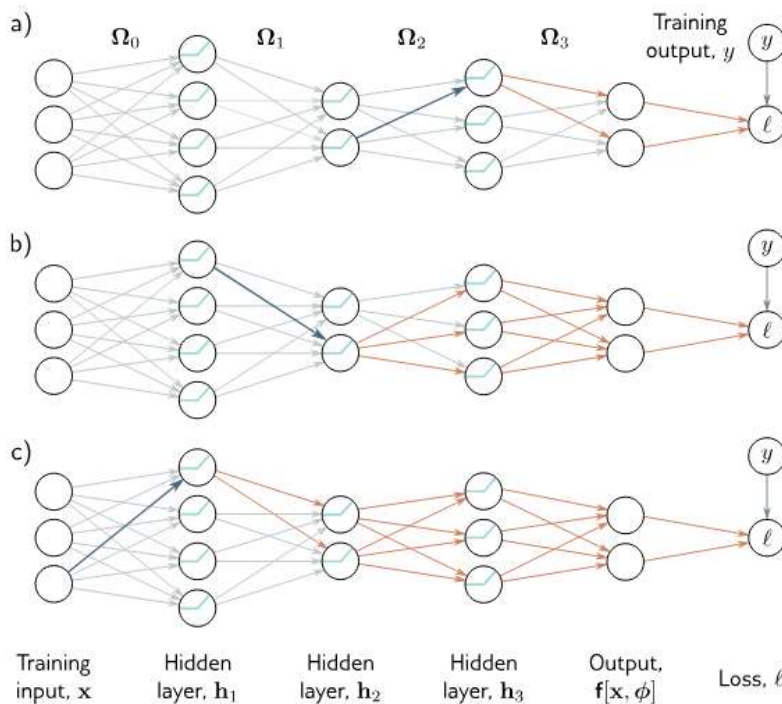


Figure 7.2 Backpropagation backward pass. a) To compute how a change to a weight feeding into layer h_3 (blue arrow) changes the loss, we need to know how the hidden unit in h_3 changes the model output f and how f changes the loss (orange arrows). b) To compute how a small change to a weight feeding into h_2 (blue arrow) changes the loss, we need to know (i) how the hidden unit in h_2 changes h_3 , (ii) how h_3 changes f , and (iii) how f changes the loss (orange arrows). c) Similarly, to compute how a small change to a weight feeding into h_1 (blue arrow) changes the loss, we need to know how h_1 changes h_2 and how these changes propagate through to the loss (orange arrows). The backward pass first computes derivatives at the end of the network and then works backward to exploit the inherent redundancy of these computations.

- Moving backward throughout the network, most the terms we need are already calculated from the previous step!
- Proceeding backwards through the network to compute the derivatives is known as the backward pass.

- Proceeding backwards through the network to compute the derivatives is known as the backward pass.

7.3: Toy Example:

- Consider a model $f(x, \theta)$ with 8 scalar parameters $\theta = \{ \beta_0, w_0, \beta_1, w_1, \beta_2, w_2, \beta_3, w_3 \}$

$$f(x, \theta) = \beta_3 + w_3 \cdot \cos[\beta_2 + w_2 \exp[\beta_1 + w_1 \sin[\beta_0 + w_0 \cdot x]]]$$

- Has the least squares loss function:

$$L[\theta] = \sum_i L_i$$

$$L_i = (f(x_i, \theta) - y_i)^2$$

- * 1 input, 1 output, 3 hidden layers, 1 hidden unit per layer
 $\cos[\cdot]$, $\sin[\cdot]$, $\exp[\cdot]$ are activation functions

- We want to compute:

$$\frac{\partial L_i}{\partial w_0}, \frac{\partial L_i}{\partial \beta_0}, \frac{\partial L_i}{\partial w_1}, \frac{\partial L_i}{\partial \beta_1}, \frac{\partial L_i}{\partial w_2}, \frac{\partial L_i}{\partial \beta_2}, \frac{\partial L_i}{\partial w_3}, \frac{\partial L_i}{\partial \beta_3}$$

- Computing by hand: hard coding is complex, doesn't exploit redundancy that we saw in backpropagation.

- Instead, we use the backpropagation algorithm. It has 2 steps:

1) Forward Pass: compute & store intermediate values & network output

Treat the loss computation as a series of calculations:

$$f_0 = \beta_0 + w_0 x_i$$

$$h_1 = \sin[f_0]$$

$$f_1 = \beta_1 + w_1 h_1$$

$$h_2 = \exp[f_1]$$

$$f_2 = \beta_2 + w_2 h_2$$

We compute & store the values of the intermediate variables f_k & h_k .

$$\begin{aligned}
 f_2 &= b_2 + w_2 h_1 \\
 h_2 &= \cos[f_2] \\
 f_3 &= b_3 + w_3 h_2 \\
 L_i &= (f_3 - y_i)^2
 \end{aligned}$$

Variables f_k & h_k !

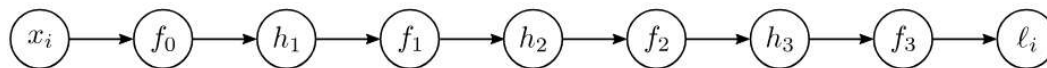


Figure 7.3 Backpropagation forward pass. We compute and store each of the intermediate variables in turn until we finally calculate the loss.

2) Backward Pass: Calculate the derivatives of each parameter, starting from the end of the network & reusing previous calculations as we move towards the start.

Backward Pass #1: Compute derivatives of L_i with respect to intermediate variables, in reverse order:

$$\frac{\partial L_i}{\partial f_3}, \frac{\partial L_i}{\partial h_3}, \frac{\partial L_i}{\partial f_2}, \frac{\partial L_i}{\partial h_2}, \frac{\partial L_i}{\partial f_1}, \frac{\partial L_i}{\partial h_1}, \frac{\partial L_i}{\partial f_0}$$

• The first is easy:

$$\frac{\partial L_i}{\partial f_3} = 2(f_3 - y_i)$$

• The next can be found with the chain rule:

$$\frac{\partial L_i}{\partial h_3} = \frac{\partial f_3}{\partial h_3} \cdot \frac{\partial L_i}{\partial f_3}$$

↳ This is very easy:
 $\frac{\partial f_3}{\partial h_3} = w_3$

↳ We already computed this above!

• We continue along the network this way, computing derivatives at the next with respect to the intermediate variables...

- We continue along the network this way, computing derivatives of the output with respect to the intermediate quantities:

$$\frac{\partial L_i}{\partial f_2} = \frac{\partial h_3}{\partial f_2} \left(\frac{\partial f_3}{\partial h_3} \frac{\partial L_i}{\partial f_3} \right)$$

$$\frac{\partial L_i}{\partial h_2} = \frac{\partial f_2}{\partial h_2} \left(\frac{\partial h_3}{\partial f_2} \frac{\partial f_3}{\partial h_3} \frac{\partial L_i}{\partial f_3} \right)$$

* This captures intuition #2

$$\frac{\partial L_i}{\partial f_1} = \frac{\partial h_2}{\partial f_1} \left(\frac{\partial f_2}{\partial h_2} \frac{\partial h_3}{\partial f_2} \frac{\partial f_3}{\partial h_3} \frac{\partial L_i}{\partial f_3} \right)$$

$$\frac{\partial L_i}{\partial h_1} = \frac{\partial f_1}{\partial h_1} \left(\frac{\partial h_2}{\partial f_1} \frac{\partial f_2}{\partial h_2} \frac{\partial h_3}{\partial f_2} \frac{\partial f_3}{\partial h_3} \frac{\partial L_i}{\partial f_3} \right)$$

$$\frac{\partial L_i}{\partial f_0} = \frac{\partial h_1}{\partial f_0} \left(\frac{\partial f_1}{\partial h_1} \frac{\partial h_2}{\partial f_1} \frac{\partial f_2}{\partial h_2} \frac{\partial h_3}{\partial f_2} \frac{\partial f_3}{\partial h_3} \frac{\partial L_i}{\partial f_3} \right)$$

* Everything in brackets was computed previously.

Backward Pass #2:

- ↳ We can now consider how loss L_i changes when we change parameters $\in \mathcal{B}_k$ and $\in \mathcal{W}_k$.
- ↳ Apply the chain rule:

$$\frac{\partial L_i}{\partial \mathcal{B}_k} = \frac{\partial f_k}{\partial \mathcal{B}_k} \underbrace{\frac{\partial L_i}{\partial f_k}}_{\text{Already computed from the first step.}}$$

$$\frac{\partial L_i}{\partial \mathcal{W}_k} = \frac{\partial f_k}{\partial \mathcal{W}_k} \underbrace{\frac{\partial L_i}{\partial f_k}}_{\text{Already computed from the first step.}}$$

- When $k > 0$

$$f_k = \mathcal{B}_k + \mathcal{W}_k h_k \Rightarrow \frac{\partial f_k}{\partial \mathcal{B}_k} = 1$$

$$\frac{\partial f_k}{\partial \mathcal{W}_k} = h_k$$

This shows intuition #1!
 The effect of a change in the weight $\mathcal{W}_k$ is proportional to the value of the source

$$\frac{\partial f_k}{\partial w_k} = h_k$$

The effect of a change in the weight w_k is proportional to the value of the source variable h_k .

• When $K=0$:

$$f_0 = \beta_0 + w_0 x_i \Rightarrow \frac{\partial f_0}{\partial \beta_0} = 1$$

$$\frac{\partial f_0}{\partial w_0} = x_i$$

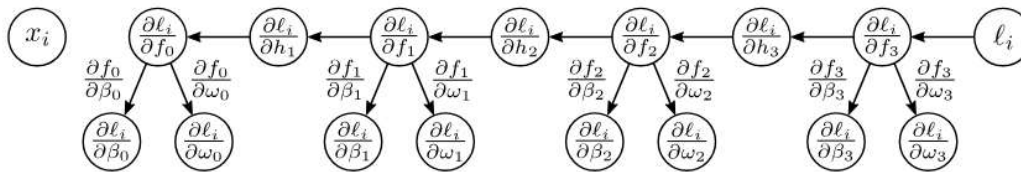


Figure 7.5 Backpropagation backward pass #2. Finally, we compute the derivatives $\partial \ell_i / \partial \beta_k$ and $\partial \ell_i / \partial w_k$. Each derivative is computed by multiplying the term $\partial \ell_i / \partial f_k$ by $\partial f_k / \partial \beta_k$ or $\partial f_k / \partial w_k$ as appropriate.

7.4: Back Propagation Algorithm:

• Repeat the process but for the 3 layer network above.

↳ Only difference is that the intermediate variables f_k, h_k are vectors.

The Biases β_k are vectors.

The Weights Ω_k are matrices.

We are using ReLU as activation function instead of sin/cos/exp

Forward Pass:

$$f_0 = \beta_0 + \Omega_0 x_i$$

$$h_1 = a[f_0]$$

$$f_1 = \beta_1 + \Omega_1 h_1$$

$$h_2 = a[f_1]$$

$$f_2 = b_2 + \Omega_2 h_2$$

$$h_3 = a[f_2]$$

$$f_3 = b_3 + \Omega_3 h_3$$

$$L_i = L[f_3, y_i]$$

* f_{k-1} represents pre-activations at the k^{th} layer

h_k contains activations at the k^{th} layer

$L[f_3, y_i]$ is an arbitrary loss function

Backward Pass #1:

Consider how loss changes when pre-activations f_0, f_1, f_2 change. Using chain rule:

$$\frac{\partial L_i}{\partial f_2} = \frac{\partial b_3}{\partial f_2} \frac{\partial f_3}{\partial h_3} \frac{\partial L_i}{\partial f_3}$$

The shapes are important. Left to right, they are: $D_3 \times D_3$ $\rightarrow$ # of hidden units in the third layer
 $D_3 \times D_f$ $\rightarrow$ Dimensionality of model output, f_3 .
 $D_f \times 1$

$$\frac{\partial L_i}{\partial f_1} = \frac{\partial h_2}{\partial f_1} \frac{\partial f_2}{\partial h_2} \left(\frac{\partial h_3}{\partial f_2} \frac{\partial f_3}{\partial h_3} \frac{\partial L_i}{\partial f_3} \right)$$

$$\frac{\partial L_i}{\partial f_0} = \frac{\partial h_1}{\partial f_0} \frac{\partial f_1}{\partial h_1} \left(\frac{\partial h_2}{\partial f_1} \frac{\partial f_2}{\partial h_2} \frac{\partial h_3}{\partial f_2} \frac{\partial f_3}{\partial h_3} \frac{\partial L_i}{\partial f_3} \right)$$

* Everything in the brackets has been previously computed

• Again, the derivatives are simple.

$\hookrightarrow \frac{\partial L_i}{\partial f_3}$ depends on the exact loss, but it's usually simple

$$\hookrightarrow \frac{\partial b_3}{\partial h_3} = \frac{\partial}{\partial h_3} [b_3 + \Omega_3 h_3] = \Omega_3^T$$

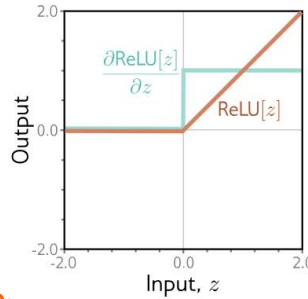
$\hookrightarrow$ Need to understand Matrix Calculus to get this result.

$\hookrightarrow \frac{\partial h_3}{\partial f_1}$ depends on the activation function.

$\hookrightarrow$ It will be a diagonal matrix, since each activation depends on the corresponding pre-activation.

$\hookrightarrow$ For ReLU:

Figure 7.6 Derivative of rectified linear unit. The rectified linear unit (orange curve) returns zero when the input is less than zero and returns the input otherwise. Its derivative (cyan curve) returns zero when the input is less than zero (since the slope here is zero) and one when the input is greater than zero (since the slope here is one).



• So for ReLU the diagonal terms in the matrix are 0 everywhere $f_z < 0$; 1 otherwise. In practice, we extract the diagonal terms in a vector by thresholding $I[f_z > 0]$; Pointwise multiply, rather than multiply by the diagonal matrix.

• As we go back through the network, we allow:

i) Multiplying by the transpose of the weight matrices W_k^T

ii) threshold based on the inputs f_{k-1} to the hidden layer. This was stored during the forward pass.

Backward Pass #2:

We know how to compute $\frac{\partial L_i}{\partial f_k}$, so we can now calculate with respect to the weights & biases.

We can use the chain rule again:

$$\begin{aligned} \frac{\partial L_i}{\partial b_k} &= \frac{\partial f_k}{\partial b_k} \frac{\partial L_i}{\partial f_k} = \frac{\partial}{\partial b_k} (b_k + W_k h_k) \frac{\partial L_i}{\partial f_k} \\ &= \frac{\partial L_i}{\partial f_k} \end{aligned}$$

↳ which was already calculated

The derivative for the weights matrix is given by:

$$\begin{aligned} \frac{\partial L_i}{\partial w_k} &= \frac{\partial f_k}{\partial w_k} \frac{\partial L_i}{\partial f_k} = \frac{\partial}{\partial w_k} (b_k + w_k h_k) \cdot \frac{\partial L_i}{\partial f_k} \\ &= \frac{\partial L_i}{\partial f_k} h_k^T \end{aligned}$$

↳ which we computed before

Backpropagation Algorithm Summary:

• Consider a deep neural network $f(x_i; \theta)$ with input x_i , has K hidden layers with ReLU activations, ; individual loss terms $L_i = L[f(x_i; \theta), y_i]$.

• The goal is to compute the derivatives $\frac{\partial L_i}{\partial \theta}$; $\frac{\partial L_i}{\partial \theta}$

- The goal is to compute the derivatives $\frac{\partial L_i}{\partial w_k}$; $\frac{\partial L_i}{\partial b_k}$ (weights ; biases)

Forward Pass:

↳ Compute ; store:

$$f_0 = b_0 + w_0 x_i$$

$$h_k = a[f_{k-1}] \quad k \in \{1, 2, \dots, K\}$$

$$f_k = b_k + w_k h_k \quad k \in \{1, 2, \dots, K\}$$

Backward Pass:

↳ Start with $\partial L_i / \partial f_K$; work backward through the network:

$$\frac{\partial L_i}{\partial b_K} = \frac{\partial L_i}{\partial f_K} \quad k \in \{K, K-1, \dots, 1\}$$

$$\frac{\partial L_i}{\partial w_k} = \frac{\partial L_i}{\partial f_k} \cdot h_k^T \quad k \in \{K, K-1, \dots, 1\}$$

$$\frac{\partial L_i}{\partial f_{k-1}} = I[f_{k-1} > 0] \odot (w_k^T \frac{\partial L_i}{\partial f_k}) \quad k \in \{K, K-1, \dots, 1\}$$

- $\odot$ denotes pointwise multiplication

$I[f_{k-1} > 0]$ is a vector containing 1s where $f_{k-1} > 0$; 0s elsewhere

Then we compute the derivatives with respect to the first set of weights ; biases:

$$\frac{\partial L_i}{\partial b_0} = \frac{\partial L_i}{\partial f_0}$$

$$\frac{\partial L_i}{\partial w_0} = \frac{\partial L_i}{\partial f_0} x_i^T$$

- These are calculated for every training example in the batch ; summed together to get the gradient for the SGD update.
- Backpropagation is computationally efficient, but not memory efficient.

Algorithmic Differentiation:

- ML Frameworks calculate derivatives automatically.
 - ↳ unlikely to code the algorithm in practice.
 - ↳ called **algorithmic differentiation**
- Each functional component [linear function, activation functions, loss functions] know how to calculate their own derivatives.
- These frameworks exploit parallelization in GPUs.
 - ↳ Its possible to perform forward + backward passes of the entire batch in parallel.
 - ↳ This means the input becomes a multi dimensional tensor.
- The idea behind backpropagation still hold when dealing with non-sequential networks / acyclic

7.5: Parameter Initialization:

- This step is crucial:

↳ Why? During the forward pass each preactivation is computed as:

$$\begin{aligned}f_k &= \beta_k + \Omega_k h_k \\ &= \beta_k + \Omega_k a[P_{k-1}]\end{aligned}$$

- Imagine we initialize all biases to 0 ; Ω to a normal distribution with mean=0, variance σ^2 .

Case #1: If variance σ^2 is very small [10^{-5}], then each element of $\beta_k + \Omega_k h_k$ is a weighted sum of h_k , where the weights are very small, so the result will be smaller than the input.

ReLU clips values < 0 , so the range of h_k will be half of f_{k-1} . This propagates to hidden layers.

Case #2: If variance is very large (10^5), each element of $\beta_k + \Omega_k h_k$ is a weighted sum of h_k , i the weights are very large.

So the result is likely much larger than the input. ReLU halves the range of inputs, but if σ^2 is large enough the magnitudes of the pre activations will still get larger. This propagates to hidden layers.

- In either case, the values of the pre activations get so small or so large it can't be represented with floating point arithmetic.

↳ **Vanishing gradient Problem**

↳ **Exploding gradient Problem**

Initialization for Forward Pass:

- Consider the computation between adjacent pre-activations f ; f' with dimensions D_n ; $D_{n'}$ respectively:

$$h = a[f]$$

$$f' = \beta + \Omega h$$

- Assume the pre activations f_j in the input layer f have variance σ_f^2 , initializing biases β_i to 0, ; weights Ω_{ij} as normally distributed with mean 0 ; variance σ_Ω^2

• We can derive expressions for mean & variance of the pre-activations f' :

Expectation is mean:

$$\begin{aligned} E[f'_i] &= E\left[b_i + \sum_{j=1}^{D_h} \omega_{ij} h_j\right] \\ &= E[b_i] + \sum_{j=1}^{D_h} E[\omega_{ij}] E[h_j] \\ &= 0 + \sum_{j=1}^{D_h} 0 \cdot E[h_j] = 0 \end{aligned}$$

↳ D_h is the dimensionality of the input layer h .

↳ Assumed distributions over h_j ; ω_{ij} are independent

• From this, we derive variance $\sigma_{f'}^2$ of the pre-activation f' :

$$\begin{aligned} \sigma_{f'}^2 &= E[(f'_i - E[f'_i])^2] \\ &= E\left[\left(b_i + \sum_{j=1}^{D_h} \omega_{ij} h_j\right)^2\right] - 0 \\ &= E\left[\left(\sum_{j=1}^{D_h} \omega_{ij} h_j\right)^2\right] \\ &= \sum_{j=1}^{D_h} E[\omega_{ij}^2] E[h_j^2] \\ &= \sum_{j=1}^{D_h} \sigma_{\omega}^2 E[h_j^2] = \sigma_{\omega}^2 \sum_{j=1}^{D_h} E[h_j^2] \end{aligned}$$

Assuming the distributions at the previous layer are symmetric about 0, half of the pre-activations will get clipped by ReLU:

$$\sigma_{f'}^2 = \sigma_{\omega}^2 \sum_{j=1}^{D_h} \frac{\sigma_{h_j}^2}{2} = \frac{1}{2} D_h \sigma_{\omega}^2 \sigma_h^2$$

• This means if we want the variance $\sigma_{f'}^2$ to be the same as the variance σ_f^2 of the original pre-activations f during the forward pass we should get:

$$\sigma_{\omega}^2 = \frac{2}{D_h} \quad \text{Dimension of the original layer to which the weights are applied.}$$

Known as He initialization

Initialization for backward pass:

• Using a similar argument we get:

$$\sigma_{\omega}^2 = \frac{2}{D_h} \quad \text{The dimension of the layer the weights feed into.}$$

Initialization for both forward & backward pass:

• If there are different numbers of hidden units in two adjacent layers then we can't satisfy both conditions above.

• Compromise: Use the mean $\frac{D_{in} + D_{out}}{2}$ as a proxy for the number of terms:

$$\sigma_a^2 = \frac{4}{D_{in} D_{out}}$$

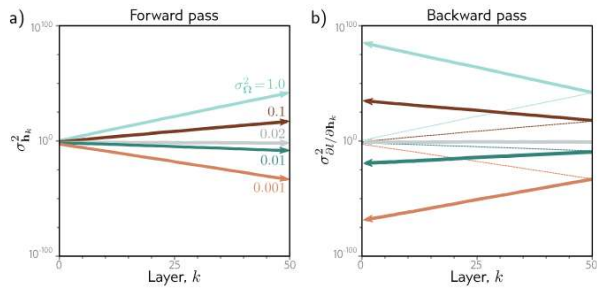


Figure 7.7 Weight initialization. Consider a deep network with 50 hidden layers and $D_h = 100$ hidden units per layer. The network has a 100-dimensional input $\mathbf{x}$ initialized from a standard normal distribution, a single fixed target $y = 0$, and a least squares loss function. The bias vectors β_k are initialized to zero, and the weight matrices Ω_k are initialized with a normal distribution with mean zero and five different variances $\sigma_{\Omega}^2 \in \{0.001, 0.01, 0.02, 0.1, 1.0\}$. a) Variance of hidden unit activations computed in forward pass as a function of the network layer. For He initialization ($\sigma_{\Omega}^2 = 2/D_h = 0.02$), the variance is stable. However, for larger values, it increases rapidly, and for smaller values, it decreases rapidly (note log scale). b) The variance of the gradients in the backward pass (solid lines) continues this trend; if we initialize with a value larger than 0.02, the magnitude of the gradients increases rapidly as we pass back through the network. If we initialize with a value smaller, then the magnitude decreases. These are known as the *exploding gradient* and *vanishing gradient* problems, respectively.

7.7: Summary:

- SGD relies on the gradient of loss with respect to the Model Parameters.
↳ Backpropagation solves this
- Gradient updates could vanish or explode, & magnitudes of the hidden unit activations can do so too.
↳ Both impede training
↳ Both can be fixed by proper initialization.

See Arora et al. (2016) for Survey of distributed training methods